M101 Spickzettel 1/2 Webauftritt erstellen & veröffentlichen

1 Web, Browser, Begriffe

- **Internet** = weltweites Netzwerk. **WWW** = ein Dienst darauf (Webseiten über HTTP), 1989 von **Tim Berners-Lee** am **CERN** erfunden, erste Website 1991.
- **HTML** = HyperText Markup Language: Auszeichnungssprache für **Struktur/Inhalt** (keine Programmiersprache). **CSS** = Cascading Style Sheets: **Darstellung**, trennt Design vom Inhalt. JavaScript = Verhalten (nicht M101).
- **Browser** (Chrome, Edge, Firefox, Safari): fordert Dateien beim **Webserver** an (HTTP/**HTTPS** = verschlüsselt), interpretiert HTML/CSS und stellt die Seite dar (rendern). Client-Server-Prinzip.
- **URL**: `https://www.zli.ch/kurse/index.html` = Protokoll · Subdomain · Domain · Top-Level-Domain (.ch) · Pfad · Datei. **DNS** übersetzt Domain → IP-Adresse.
- **Webstandards**: von **W3C** und **WHATWG**; Ziel: gleiche Darstellung in allen Browsern, Barrierefreiheit, Benutzerfreundlichkeit.
- **Endgeräte** (Desktop/Tablet/Smartphone) unterscheiden sich in Auflösung, Bildschirmgrösse, Bedienung (Maus/Touch) und Bandbreite → kleine Bilder, `viewport`-Meta, flexible Layouts.

2 Projektablauf (Handlungsziele)

1. **Analyse**: Ziel, Zielgruppe, Inhalte, Funktionen der Vorgabe.
2. **Sitemap** = Struktur: welche Seiten, welche Inhalte wo, Hierarchie (M101: max. 3 Ebenen). **Mockup/Wireframe** = Layoutskizze inkl. Navigation (z. B. Adobe XD, Figma).
3. **Umsetzen** mit HTML + CSS.
4. **Testen** gegen die Vorgabe (Validator, Testprotokoll, Browser/Geräte).
5. **Veröffentlichen** auf Webserver (FTP).

3 HTML-Grundgerüst

```
<!DOCTYPE html>            ← HTML5
<html lang="de">
<head>                     ← unsichtbar
  <meta charset="utf-8"> ← Umlaute!
  <meta name="viewport" content=
   "width=device-width, initial-scale=1.0">
  <meta name="description" content="...">
  <meta name="author" content="...">
  <title>Titel im Tab</title>
  <link rel="stylesheet" href="css/style.css">
  <link rel="icon" href="favicon.png">
</head>
<body> ... sichtbarer Inhalt ... </body>
</html>
```

Aufbau Tag: `<a href="x.html">Text</a>` = Start-Tag · Attribut**name**="**wert**" · Inhalt · End-Tag. Element = Start-Tag + Inhalt + End-Tag. **Leere Elemente** ohne End-Tag: `img br hr meta link input` (`<br>` oder `<br />`). Verschachtelung: zuletzt geöffnet = zuerst geschlossen. Kommentar: `<!-- ... -->`.

Sonderzeichen: `&` & · `<` `>` < > · ` ` festes Leerzeichen · `©` © · `—` — · `"` "

4 Semantische Elemente (HTML5)

`header`	Kopf: Logo, Titel, Navigation
`nav`	Navigationslinks (mit `ul`/`li`)
`main`	Hauptinhalt, **nur 1× pro Seite**
`section`	thematischer Abschnitt (mit Überschrift)
`article`	eigenständiger Inhalt (Beitrag, Karte)
`aside`	Randbemerkung, Seitenleiste
`footer`	Fuss: Copyright, Impressum, Kontakt
`figure`/`figcaption`	Bild mit Bildunterschrift
`div` / `span`	ohne Bedeutung, nur für CSS (block / inline)

Warum Semantik? Suchmaschinen (SEO) gewichten Inhalte nach Tags (`title`, `h1` → Keywords), **Screenreader** lesen Sehbehinderten den Code vor (Barrierefreiheit), Code lesbarer. Ein `h1` pro Seite, Überschriften nicht überspringen.

5 Wichtige Tags & Attribute

`h1`–`h6`	Überschriften (Hierarchie)
`p` · `br` · `hr`	Absatz · Zeilenumbruch · Trennlinie
`strong` / `em`	wichtig (fett) / betont (kursiv); `b`/`i` nur optisch
`sup` `sub`	hoch- / tiefgestellt (Fussnote*, H_2O)
`blockquote` · `q`	Zitat als Block (`cite="URL"`) · inline
`a href`	Link, `target="_blank"` = neues Tab
`img src alt`	Bild + Alternativtext (Pflicht)
`id="x"`	eindeutig, 1× pro Seite (CSS `#x`, Anker)
`class="x"`	mehrfach verwendbar (CSS `.x`)
`style="..."`	Inline-CSS (vermeiden)
`title="..."`	Tooltip beim Hover

Block (neue Zeile, volle Breite): `h1–h6 p div ul ol li table form header main footer section nav`. **Inline** (nur so breit wie nötig, kein Umbruch): `span a img em strong br`.

6 Links & Pfade

`kontakt.html` / `./kontakt.html`	gleicher Ordner
`seiten/team.html`	Unterordner
`../index.html`	ein Ordner höher
`/kontakt.html`	ab DocumentRoot (Server-Wurzel)
`#team`	Anker → Element mit `id="team"`
`seite.html#team`	Anker auf anderer Seite
`https://www.zli.ch/x.html`	**absolut** (mit Protokoll!)
`www.zli.ch/x.html`	**nicht** absolut (https fehlt → wird relativ)
`C:\User\...\x.html`	absolut auf Festplatte → **nie**! online kaputt
`mailto:a@b.ch` · `tel:+41...`	E-Mail · Telefon

Relativ = funktioniert nach Hosting-Wechsel weiter (gleiche Ordnerstruktur). Extern = absolut.

7 Datei- & Ordnernamen

- **alles klein** (Windows: `Bild.jpg` = `bild.jpg`; Linux-Webserver: zwei verschiedene Dateien!)
- **keine Leerzeichen, Umlaute, Sonderzeichen** → `-` oder `_`: `ueber-uns.html`
- Startseite heisst **`index.html`**; mit Ordnern arbeiten (`css/`, `bilder/`); Sicherheitskopien.
- Falsch: `Seite 2 (Über uns).html`, `erste Seite Style.css`, `Favicon.png`

8 Listen & Tabellen

```
<ul><li>Punkt</li></ul>          ungeordnet
<ol start="5"><li>..</li></ol>   nummeriert
<dl><dt>Begriff</dt><dd>Erklärung</dd></dl>
```

Verschachtelt: neue `ul` **innerhalb** eines `li`. Navigation immer `nav` + `ul`.

```
<table>
 <thead><tr><th>Tag</th><th>Zeit</th></tr>
</thead>
 <tbody><tr><td colspan="2">zu</td></tr></tbody>
</table>
```

`tr` Zeile · `th` Kopfzelle · `td` Datenzelle · `thead`/`tbody`/`tfoot` Struktur · `caption` Titel. **colspan** = Spalten verbinden (horizontal), **rowspan** = Zeilen verbinden (vertikal). Zellen pro Zeile (inkl. colspan) = Spaltenzahl. CSS: `border-collapse: collapse` = einfache Rahmen.

9 Bildformate

	JPG	PNG	GIF	SVG
Art	Raster	Raster	Raster	**Vektor**
Kompr.	hoch, **verlust-behaftet**	verlust**frei**	gering	keine
Farben	16,7 Mio.	sehr viele	**max. 256**	16,7 Mio.
Transp.	nein	ja (Alpha)	ja (1 Farbe)	ja
Anim.	nein	ja (.apng)	ja	ja
Grösse	sehr klein	klein–gross	gross	individuell
Ideal	**Fotos**	Logos, Icons, Transparenz	Animation, Clip-Art	Logos, Icons, Diagramme

Raster (Pixel) wird beim Vergrössern unscharf; **Vektor** (Formen/Pfade) verlustfrei skalierbar. BMP: nicht verwenden (riesig). WebP: modernes Webformat (klein, Transparenz). **Bilder fürs Web**: auf Anzeigegrösse verkleinern, komprimieren, sprechender Dateiname, immer `alt`. **Favicon**: quadratisch, 32×32 px, `<link rel="icon" type="image/png" sizes="32x32" href="favicon-32x32.png">`. **PDF** webgerecht: komprimiert, Link mit Typ+Grösse: „preise.pdf – 56 kB".

10 Video & Audio

```
<video src="film.mp4" controls poster="v.jpg">
  <source src="film.webm" type="video/webm">
</video>
<audio controls>
  <source src="ton.mp3" type="audio/mpeg">
  <source src="ton.ogg" type="audio/ogg">
</audio>
```

Attribute: `controls` Player sichtbar · `autoplay` automatisch (browserabhängig) · `loop` Endlosschleife · `muted` stumm · `poster` Vorschaubild. `source` = Alternativformate. **Video**: **MP4** Standard, gute Kompression, höchste Kompatibilität · **WebM** offen (Google) · **OGV** offen, wenig verbreitet. **Audio**: **MP3** verlustbehaftet, klein, verbreitet · **AAC** besser als MP3, Apple-Standard · **OGG** offen · **WAV** unkomprimiert, sehr gross.

11 Urheberrecht, Bildrechte, Datenschutz

- Für jede Verwendung eines fremden Werks braucht es eine **Erlaubnis (Lizenz)** des Rechteinhabers. Immer!
- Geschützt: Werke = **geistige Schöpfung mit individuellem Charakter** (Texte, Musik, Bilder, Fotos, Filme, Software, Design). **Nicht** geschützt: Ideen, Konzepte, Methoden, reines Handwerk.
- Schutz entsteht **automatisch bei Erschaffung**, keine Anmeldung/Register. © hat in der CH keine rechtliche Wirkung, nur Hinweis (empfohlen).
- Dauer: **70 Jahre nach Tod** des Urhebers, Software **50 J.**; verwandte Schutzrechte (Interpreten, CD) 50 J. Gesetz: URG; zuständig: **IGE**.
- **Privatgebrauch** erlaubt (inkl. Download/Streaming), **Hochladen/Veröffentlichen nicht** (auch ohne Gewinn). **Zitat** erlaubt: nicht länger als nötig, als Zitat kennzeichnen, Quelle nennen.
- **Recht am eigenen Bild**: abgebildete Person entscheidet über Aufnahme und Veröffentlichung → Einwilligung einholen (Ausnahme: überwiegendes öffentliches Interesse, z. B. Menge an Veranstaltung). Im Zweifel fragen.
- **Creative Commons**: BY = Namensnennung · NC = nicht kommerziell · ND = keine Bearbeitung · SA = Weitergabe unter gleichen Bedingungen · CC0 = frei. Gratis-Bildportale (Unsplash, Pixabay, Pexels): Bedingungen lesen, Urheber + Link nennen. Stock (iStock, Shutterstock): kostenpflichtige Lizenz.
- **KI-Bilder**: ohne menschliche Schöpfung meist kein urheberrechtlicher Schutz; Nutzungsbedingungen des Tools beachten, keine geschützten Werke/echten Personen nachbilden.
- **Datenschutz** (DSG): Personendaten nur nötig & zweckgebunden; Website mit Impressum und Datenschutzerklärung.

12 Testen & Barrierefreiheit

W3C Validator prüft Syntax gegen den Standard: HTML `validator.w3.org`, CSS `jigsaw.w3.org/css-validator`.

Testprotokoll (was, wie, Resultat, Massnahme). *Technisch*: Syntax HTML/CSS, DOCTYPE, Meta-Angaben (Autor, Beschreibung), nur von allen Browsern unterstützte Technologien, Bilder (Auflösung, Grösse, Format), Aktualität, Namenskonventionen. *Gestaltung*: einheitliche Schrift, Ausrichtung, Lesbarkeit/Kontrast, Bedienelemente (Links, Buttons), Layout/CI, HTML-fremde Inhalte (Typ+Grösse). In mehreren Browsern + Geräten testen (F12 Gerätemodus), alle Links klicken.

Barrierefreiheit: Textalternativen (`alt`), Kontraste & Farben, Skalierbarkeit (Zoom), Linearisierbarkeit (Reihenfolge im Code sinnvoll), Tastaturbedienung/Fokus, ARIA, Verständlichkeit & Navigation, strukturierte Inhalte (Semantik). Tools: Silktide-Extension, Accessibility-Checkliste (a11y.digitaldialog.swiss).

13 Veröffentlichen

- **Webhosting** = Speicherplatz auf einem Webserver (+ Domain), z. B. Hostpoint.
- **FTP** (File Transfer Protocol) überträgt Dateien zwischen Rechnern, unabhängig von Betriebssystem; auch Ordner anzeigen, umbenennen, löschen, Zugriffsrechte setzen.
- Zugang: Server/Host, Benutzername, Passwort, Port (FTP 21). **Sicher**: **SFTP** (Port 22) / FTPS – normales FTP ist unverschlüsselt.
- Werkzeuge: FTP-Client (**FileZilla**, **Cyberduck**), Browser-Add-on (fireFTP), **WebFTP** des Anbieters (umständlicher).
- Vorher: `index.html` im Hauptordner, relative Pfade, Dateinamen klein, Bilder optimiert, validiert. Hochladen in Web-Ordner (z. B. `public_html`/`httpdocs`), danach **online testen**.

M101 Spickzettel 2/2 CSS

14 Einbindung · 15 Selektoren · 16 Spezifität · 17 Box-Modell · 18 Display/Position · 19 Text/Farben · 20 Flex · 21 Grid · 22 Fehler · 23 Fragen

14 CSS-Aufbau & Einbindung

```
h1, h2 {            ← Selektor
  color: #2a1959;    ← Deklaration
  font-size: 1.5em;     (Eigenschaft: Wert;)
}  /* Kommentar */
```

Art	Code	+ / –
Inline	`<p style="color:red">`	+ schnell, einzeln · – unübersichtlich, keine Trennung, schwer wartbar
Intern	`<style>...</style>` im head	+ zentral · – gilt nur für 1 Seite
Extern	`<link rel="stylesheet" href="style.css">`	+ Trennung, 1 Datei für alle Seiten, wartbar · – 1 Datei mehr laden
@import	`@import url("x.css");`	+ CSS aufteilen · – lädt langsamer

Kaskade („Cascading"): bei Konflikt entscheidet 1. `!important`, 2. Spezifität, 3. Reihenfolge (später gewinnt). Inline > intern/extern. **Vererbung**: Text-Eigenschaften (`color, font-*, line-height`) erben Kinder; `margin, padding, border, width, background` nicht.

15 Selektoren

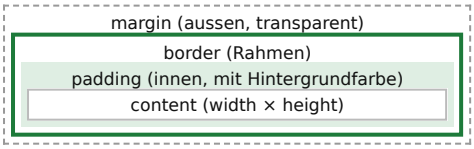
`*`	alle Elemente
`p`	Element(typ)
`.klein`	Klasse `class="klein"`
`#intro`	ID `id="intro"`
`h2, p`	Gruppe: h2 **und** p
`nav a`	Nachfahre: alle a irgendwo in nav
`ul > li`	direktes Kind
`h2 + p`	p direkt nach h2
`a.aktiv`	a mit Klasse aktiv (ohne Leerzeichen!)
`[type="email"]`	Attribut
`a:link :visited :hover :active`	unbesucht · besucht · Maus drüber · beim Klick (Reihenfolge **LVHA**)
`:focus`	per Tastatur/Klick ausgewählt
`li:first-child :last-child`	erstes/letztes Kind
`tr:nth-child(even)`	jedes 2. (odd, 3, 2n+1)
`p::first-line ::before`	Pseudo-Element

16 Spezifität (wer gewinnt?)

Selektor	Wert
`style="..."` (inline)	1-0-0-0
`#id`	0-1-0-0
`.klasse, :hover, [attr]`	0-0-1-0
`p, ::before`	0-0-0-1
`*`	0

Zusammenzählen, von links vergleichen: `#nav a` (0-1-0-1) > `.menu a` (0-0-1-1) > `a`. Gleich → die **weiter unten** stehende Regel gewinnt. `!important` schlägt alles (vermeiden). Im Browser F12 → Styles: durchgestrichen = überschrieben.

17 Box-Modell



Breite (Standard `content-box`) = width + padding links/rechts + border links/rechts (+ margin). Mit `box-sizing: border-box` sind padding + border **in** width enthalten. Bsp: width 200 + padding 2×10 + border 2×1 = **222 px** (border-box: 200).
Kurzschreibweise: `margin: 10px` alle · `10px 20px` oben/unten · links/rechts · `5px 10px 15px` oben · l/r · unten · `1px 2px 3px 4px` oben rechts unten links (Uhrzeiger). `border: 1px solid red` (Breite Stil Farbe; Stil: solid dashed dotted double none). `border-radius: 5px` runde Ecken. `margin: 0 auto` + width = Block zentrieren. Vertikale margins fallen zusammen (der grössere zählt). Browser-Standards: body 8px margin, ul padding-left.

18 display, position, float

`block`	neue Zeile, volle Breite, width/height wirken
`inline`	im Textfluss, nur so breit wie Inhalt, **kein** width/height/margin oben-unten
`inline-block`	im Fluss nebeneinander, aber width/height möglich (z. B. Nav-`li`)
`none`	unsichtbar, kein Platz (`visibility:hidden`: unsichtbar, Platz bleibt)
`flex` / `grid`	Layout-Container für die Kinder

`static`	Standard, kein top/left
`relative`	verschoben relativ zur normalen Position, Platz bleibt; Bezugspunkt für absolute Kinder
`absolute`	aus dem Fluss, relativ zum nächsten positionierten Vorfahren
`fixed`	relativ zum Viewport (Fenster), bleibt beim Scrollen
`sticky`	normal, „klebt" ab Scroll-Position (`top:0`)

Mit `top right bottom left; z-index` = Stapelreihenfolge (höher = vorne, nur positionierte). `float: right` → Element rechts, Text fliesst herum; `clear: both` beendet. `overflow: hidden/auto` = Überlauf abschneiden/scrollen.

19 Einheiten, Farben, Text, Hintergrund

Einheiten: `px` absolut · `%` vom Eltern-Element · `em` × Schriftgrösse des Elternelements (1.5em = 1,5-fach) · `rem` × Schriftgrösse von `html` (meist 16px) · `vw/vh` % der Fensterbreite/-höhe · `fr` Anteil (Grid). Zahl und Einheit **ohne Leerzeichen**; bei 0 keine Einheit nötig.
Farben: Name `red` · Hex `#rrggbb` (`#ff0000`), kurz `#f00, #030` = `#003300` · `rgb(255,0,0)` · `rgba(0,0,0,0.5)` (a = Deckkraft 0–1) · `hsl(0,100%,50%)`. `color` = Schrift, `background-color` = Hintergrund.

`font-family`	`Verdana, Arial, sans-serif` (immer Ersatz: serif, sans-serif, monospace)
`font-size`	`16px, 1.2em, small`
`font-weight`	`bold, normal`, 100–900
`font-style`	`italic`
`font-variant`	`small-caps` = Kapitälchen
`text-transform`	`uppercase` / `lowercase` / `capitalize`
`text-decoration`	`none` (Link ohne Unterstrich), `underline`
`text-align`	`left center right justify`
`line-height` · `letter-spacing`	Zeilenhöhe `1.5` · Zeichenabstand
`list-style`	`none, square, decimal`

Hintergrundbild: `background-image: url("../bilder/x.jpg")` (Pfad relativ zur **CSS-Datei**) · `background-repeat: repeat | repeat-x | repeat-y | no-repeat` · `background-position: right 100px | center | top left` · `background-size: cover` · `background-attachment: fixed` (bleibt beim Scrollen).

Gestaltung: max. 2–3 Schriften, wenige einheitliche Farben (Corporate Identity), genug **Kontrast** (dunkel auf hell), serifenlos am Bildschirm gut lesbar, Weissraum.

20 Flexbox (1 Richtung)

Auf den **Eltern-Container**: `display: flex` → direkte Kinder nebeneinander. Hauptachse = `flex-direction`, Querachse senkrecht dazu.

Container	
`flex-direction`	`row` (Std.) | `column` | `row-reverse`
`justify-content`	Hauptachse: `flex-start center flex-end space-between space-around space-evenly`
`align-items`	Querachse: `stretch` (Std.) `center flex-start flex-end`
`flex-wrap`	`wrap` = Umbruch erlauben
`gap`	Abstand zwischen Items
Item (Kind)	
`flex: 1`	wächst, teilt Platz gleich (`flex: 2` = doppelt)
`flex: 0 0 300px`	grow shrink basis: fix 300px
`order` · `align-self`	Reihenfolge · eigene Querausrichtung

```
nav ul { display:flex; gap:20px;
         list-style:none; }
/* Logo links, Nav rechts */
header { display:flex; align-items:center;
         justify-content:space-between; }
/* Inhalt in der Box mittig */
.box { display:flex; justify-content:center;
       align-items:center; }
```

21 Grid (2 Richtungen)

```
#galerie {
  display: grid;
  /* 4 gleich breite Spalten */
  grid-template-columns: repeat(4, 1fr);
  grid-template-rows: 200px auto;
  grid-auto-rows: 200px; /* weitere */
  gap: 1rem;             /* Abstand */
}
.gross { grid-column: span 2;
         grid-row: span 2; }
/* von Linie 1 bis 3 */
.kopf { grid-column: 1 / 3; }
/* ganze Breite */
.x { grid-column: 1 / -1; }
```

`1fr 2fr` = Anteile; `200px 1fr` = fix + Rest. Linien zählen ab 1. `grid-template-areas: "kopf kopf" "nav main";` + `grid-area: kopf;`. Bild füllt Zelle: `width:100%; height:100%; object-fit:cover;`. **Flex vs. Grid**: Flex = eine Reihe/Spalte (Nav, Karten); Grid = Zeilen **und** Spalten (Galerie, Seitenraster).

22 Häufige Fehler (vgl. Übung 3.3)

Symptom	Ursache
gar kein Design	`link`-Pfad/Name falsch, `rel="stylesheet"` fehlt
Bild fehlt	Pfad/Endung/Gross-Klein falsch, `"` fehlt
Text verschwindet	`<h2Titel` (`>` fehlt), `<!--` nicht zu
ab da alles kursiv/fett	`em/strong/a` nicht geschlossen
ä ö ü kaputt	`meta charset="utf-8"` fehlt
Anker springt nicht	`#kontakt` ≠ `id="Kontakt"`
Link nur lokal ok	`C:\...` statt relativ
Tabelle verschoben	colspan/Zellenzahl falsch
1 Eigenschaft wirkungslos	Tippfehler `colour widht, 10 px`, falscher Wert `display:flexbox`
2 Zeilen wirkungslos	`;` fehlt
ab da wirkt nichts	`}` fehlt
Regel greift nicht	`.x` vs `#x` verwechselt; Spezifität
Flex wirkt nicht	auf Kind statt Eltern gesetzt
horiz. Scrollbalken	feste px-Breite, Bild ohne `max-width:100%`

23 Typische Fragen – Kurzantwort

id vs. class	id eindeutig (1× pro Seite, `#`), class mehrfach (`.`)
margin vs. padding	margin = Aussenabstand (ausserhalb Rahmen, transparent); padding = Innenabstand (innerhalb, hat Hintergrundfarbe)
block vs. inline	block: neue Zeile, volle Breite; inline: im Textfluss, nur Inhaltsbreite
relativ vs. absolut	relativ: ab aktueller Datei (`../`), bleibt bei Umzug gültig; absolut: komplette URL mit `https://`
JPG vs. PNG	JPG Fotos, verlustbehaftet, klein; PNG Grafiken, verlustfrei, Transparenz
Raster vs. Vektor	Pixel, unscharf beim Vergrössern; Formen, beliebig skalierbar (SVG)
Warum externes CSS?	Trennung Inhalt/Design, eine Datei für alle Seiten, einfach wartbar
Warum `alt`?	Screenreader (Barrierefreiheit), Ersatz wenn Bild nicht lädt, SEO
Flex vs. Grid	Flex eindimensional (Reihe od. Spalte), Grid zweidimensional
FTP vs. SFTP	FTP unverschlüsselt (Port 21), SFTP verschlüsselt (Port 22)
Validator wozu?	prüft Code gegen W3C-Standard → gleiche Darstellung in allen Browsern
HTML vs. CSS	HTML = Struktur/Inhalt (Auszeichnungssprache), CSS = Aussehen

Nur am Rand im Kurs: Formular `<form><label for="n">Name</label><input type="text" id="n" name="n"><button>Senden</button></form>` (Typen: text, email, password, checkbox, radio, submit). Responsive: `viewport`-Meta + `@media (max-width: 700px) { ... }` = Regeln nur für schmale Bildschirme.